

Assignment 3

1. Assignment Description

Problem: L-System (50 points)

In this assignment, you are required to implement an L-System (Lindenmayer system) using C++. This system will facilitate the procedural generation of structures, providing functionality for applying string rewriting rules and parsing turtle graphics instructions.

How the L-System Works

The generation process consists of two main phases:

1. String Generation: The system starts with an initial string called the "Axiom". For a specified number of "iterations", every character in the string is simultaneously evaluated. If a character matches a defined "Rule" (e.g., $F \rightarrow F+F-F$), it is replaced by the corresponding string. Characters with no matching rules remain unchanged.
2. Interpretation: After all iterations are complete, the program reads the final generated string character by character and translates it into drawing commands.

L-System Operations and Symbols

Your program must process an initial string over a specified number of iterations, then interpret the resulting commands using the following six specific symbols:

- F: Move forward by the given step length and draw a line segment.
- f: Move forward by the given step length without drawing a line.
- +: Turn left by increasing the current angle by the specified angle amount.
- -: Turn right by decreasing the current angle by the specified angle amount.
- [: Save the current state (X coordinate, Y coordinate, and angle).
-]: Restore the most recently saved state that has not yet been restored, setting it as the current position and angle.

Note

The initial state of your L-System must start at the origin, coordinates (0.0, 0.0), facing an initial angle of 0.0 degrees. This angle represents pointing straight right (i.e., moving along the positive X-axis, where $x > 0$ and $y = 0$).

String Rewriting Example

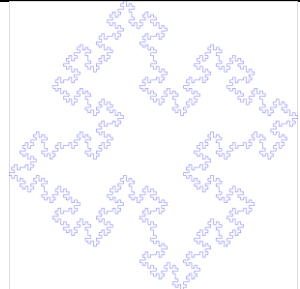
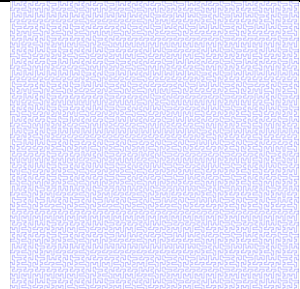
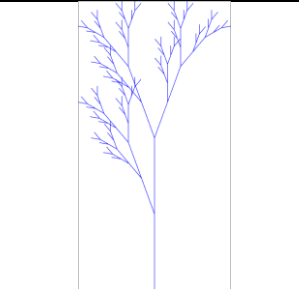
To better understand the string generation phase, consider the following simple L-System parameters:

- Axiom: F
- Rule: $F \rightarrow F+F-F$
- Iterations: 2

The string evolves step-by-step as follows:

- Iteration 0: F
(This is simply the initial Axiom. No rules have been applied yet.)
- Iteration 1: F+F-F
(The single 'F' from Iteration 0 is replaced by 'F+F-F'.)
- Iteration 2: F+F-F+F+F-F-F+F-F
(Every 'F' from Iteration 1 is simultaneously replaced by 'F+F-F'. The '+' and '-' symbols remain unchanged because there are no rewriting rules defined for them.)

Visual Examples of L-Systems

Result			
Axiom	F+F+F+F	-YF	X
Iterations	3	4	6
Angle	90	90	20
Roles	F→F+F-F-FF+F+F-F	X→XFX-YF- YF+FX+FX-YF- YFFX+YF+FXFXYF- FX+YF+FXFX+YF- FXYF-YF- FX+FX+YFYF-	F→FF
		Y→+FXFX-YF- YF+FX+FXFYF+FX- YFYF-FX- YF+FXFYFYF-FX- YFFX+FX+YF-YF- FX+FX+YFY	X→F[+X]F[-X]+X

Input

- The first line contains an integer N , representing the number of test cases. Each test case consists of multiple lines detailing the L-System parameters:
 - axiom: A string representing the initial seed state.
 - iterations: An integer indicating how many times the replacement rules should be recursively applied.
 - step_length and angle: Two floating-point numbers separated by a space, dictating movement distance and turning degrees.
 - n_rules: An integer representing the number of replacement rules.
 - Rules: The next n_rules lines each contain a character `symbol` and a string replacement, separated by a space.
- Constraints
 - number of test cases N : $1 \leq N \leq 20$.
 - iterations: $0 \leq iterations \leq 8$.
 - step_length: $0.0 < step_length \leq 100.0$.
 - angle: $0.0 \leq angle \leq 360.0$ in degrees.
 - n_rules: $0 \leq n_rules \leq 10$.
 - axiom Length: $1 \leq \text{length of axiom} \leq 200$ characters.
 - Rules Replacement Length: $1 \leq \text{length of each replacement string} \leq 50$ characters.

- Character Set: All characters used in the `axiom`, `rule symbol`, and `replacement` strings are standard printable ASCII characters.

Output

- For each test case, simulate the L-system and output all drawn line segments to the terminal.
- Each output line must contain exactly 2 points representing a line segment (start and end) in the format: `x1,y1,x2,y2`. (You can round coordinates to 2 decimal places for better readability.)

Sample Input

```
1
F
1
10.0 90.0
1
F F+F
```

Sample Output

```
0.00,0.00,10.00,0.00
10.00,0.00,10.00,10.00
```

2. Submission Guidelines

Please organize your code and files according to the following directory structure and compress them into a single `.zip` archive before uploading.

- Single Submission Archive (`.zip`): Place your main function code (`main.cpp`), all custom `.cpp` and `.h` files, and any other files required for compilation into a specific root folder. Compress this folder into a `.zip` file. (Note: If you organize your source codes into multiple subdirectories, you must include a `Makefile` in the root of your folder.)
- The `build` Directory: You must create a subfolder named `build` inside your root directory. This folder must contain your compiled executable file (`.exe`). Any required dependencies (e.g., `.dll` files) and generated graphical output files (e.g., `.svg`, `.png`, `.jpg` for the bonus) should also be placed or generated in this folder.

Naming Example & Directory Structure:

- Zip File Name: `1142DS_<Student ID>_a3.zip`
(e.g., `1142DS_B112345678_a3.zip`)

Your `.zip` archive must follow this file structure:

```
1142DS_B123456789_a3.zip/
├── build/ # Must contain executable
│   ├── 1142DS_B123456789_a3.exe
│   └── ... # dependencies (.dll) / outputs (.png / .svg)
├── main.cpp
└── ... # And any other .cpp/.h/Makefile files
```

3. Grading Policies

General Criteria (15 points):

- Coding Style (10 points): Assessed based on code readability, consistent indentation, and meaningful naming conventions. Arbitrary or meaningless variable names (e.g., a, b, c, aa, bb, cc) are strictly prohibited and will result in point deductions.
- Comments (5 points): Assessed based on brief and clear explanations of your function purposes and the overall processing flow. Focus on explaining the core logic. Do not add trivial or redundant comments for basic C++ syntax (e.g., commenting on `#include` or `using namespace std`). Additionally, every source file must include a file header specifying the author (including student ID), the date of writing (e.g., Mar. 22, 2026), and a description of the program's functionality and usage.

Problem-Specific Criteria (35 points)

- Correctness (30 points): Points are awarded based on the percentage of test cases your program successfully passes.
- Bonus (5 points): In addition to the terminal text output, implement a graphical rendering of your L-System results. You can achieve this by having your program write the line coordinates into an image file (e.g., .svg, .jpg, .png format) or by drawing the lines directly on the screen using a graphical user interface (GUI) library (such as [QT](#), [FLTK](#), [ImGUI](#), etc.).

4. Restrictions (Not Allowed)

Unless explicitly permitted by the assignment prompt, the usage of container libraries in the C++ Standard Template Library (STL)—such as `vector`, `list`, `map`, `set`, `deque`, or similar container structures—is strictly prohibited. If such libraries are used, the correctness score will be automatically **marked as 0**. (Note: You are permitted to use `string` for standard string manipulation)